

Board Game Recommendation and Strategy Explorer

CIS 5500 Final Report — Spring 2026

Yanru Fang

Yuming Li

Yunya Huang

Fusang Leng

Name	Penn email	GitHub
Yanru Fang	fang13@seas.upenn.edu	Yanru-Fang
Yuming Li	ming30@seas.upenn.edu	Yming30
Yunya Huang	hyunya22@seas.upenn.edu	hyunya22
Fusang Leng	lengfs@seas.upenn.edu	fusangleng2003

1 Project description

The Board Game Recommendation and Strategy Explorer is a search and analytics web app for board games. It combines two BoardGameGeek (BGG) datasets — a game catalog and a community-review export — into one PostgreSQL schema, then exposes both browsing-style and analytical access on top of it.

A user can:

- **search** the catalog by name, release year, supported player count, and minimum rating;
- open a **game detail page** with the description, key facts, the distribution of community ratings, the most recent reviews, and a list of similar games;
- explore four data-driven **insights** (the four “complex” queries) that surface non-obvious patterns: games that beat their year’s average; games whose reviews disagree with the catalog rating; dormant top-rated games; and globally outperforming games.

The app is deliberately scoped to be *useful* within a domain we know well (board games) rather than aiming for breadth — the value comes from the analytical insights rather than a long catalog.

2 System architecture

```
React + Vite --HTTP--> Express API (Node 20, localhost:8080) --pg--> PostgreSQL (AWS RDS)
      |
      v
lru-cache (in-process, 24 h TTL)
```

- **Database** — PostgreSQL on AWS RDS (db.t3.micro). Two business tables (`Game`, `Review`), two raw staging tables, and one materialized per-game review summary (`game_review_stats`); B-tree + `pg_trgm` GIN indexes on the columns used by the slow queries. Schema in `sql/schema.sql`, summary view in `sql/materialized_views.sql`, indexes in `sql/indexes.sql`.
- **Backend** — Node.js 20 + Express. 13 read-only routes, all queries go through a connection pool (`pg`) and an LRU cache. `backend/server.js`.

- **Frontend** — React 18 + Vite, served from Vite’s dev server on `localhost:5173`. Five pages (Home, Search, Game detail, Insights, About). The API base URL is a build-time env var (`VITE_API_BASE_URL`) that defaults to `http://localhost:8080`.

3 Data sources

The application draws on two BoardGameGeek (BGG) datasets published on Kaggle. After cleaning and entity resolution (see §4), they populate the two business tables in the schema.

Source	Used for	Link
BoardGameGeek games + ranking dataset (Kaggle)	One row per game: title, year, BGG metrics (avg rating, geek rating, voters, rank).	https://www.kaggle.com/datasets/gabrio/board-games-dataset
BoardGameGeek re-views dataset (Kaggle)	One row per individual user review/rating, with timestamps.	https://www.kaggle.com/datasets/jvanelteren/boardgamegeek-reviews

Summary statistics after load

- **Game: 126,266 rows**, 16 attributes per row (`game_id`, `name`, `description`, `year`, `player/-playtime/age` ranges, two image URLs, four BGG metrics, `complexity`). `avg_rating` and `geek_rating` are 0–10 floats; `rank` is sparse (only ranked games have a value); BGG-published `num_voters` ranges roughly from 0 to ~110,000 in the dataset.
- **Review: 29,592,656 rows** (M5 reload — the M4 measurements earlier in this repo were taken on a 9.28 M-row sample of the same source, ~3.2× smaller). 8 attributes per row; `rating` is a 0–10 float; `post_date` is set on ~99.95% of rows (the rest have a NULL `post_date` — handled in the R5 query, see §10).

4 Data cleaning and ingestion

Loading is a three-step pipeline:

1. **Stage** the raw CSVs into `raw_boardgames_csv` and `raw_reviews_csv` via `psql \copy`. Schema mirrors the CSV column order so no rewrite is needed.
2. **Clean + insert into Game** (`sql/load_game.sql`):
 - drop rows with missing `game_id` or empty `title`;
 - `NULLIF(TRIM(...), '')` to normalize empty strings to NULL;
 - dedupe on `game_id` with `DISTINCT ON`, keeping the row with the lowest BGG `rank` (highest-quality canonical entry).
3. **Clean + insert into Review** (`sql/load_review.sql`):
 - parse the two timestamp columns with explicit format strings (`TO_TIMESTAMP(..., 'YYYY/MM/DD HH24:MI')`);
 - parse `post_date` with `TO_DATE(postdate, 'YYYY/MM/DD')`, which reads the leading date

prefix and ignores any trailing time component (the CSV stores ISO-like strings with mixed precision);

- dedupe on `review_id` (BGG can re-emit a review on rating updates);
- inner-join against `Game(game_id)` so any review pointing at a dropped/missing game is excluded — preserves referential integrity.

Per the M2 plan, we additionally treat the union of the two BGG game exports (Datasets 1 and 2 from the proposal) as a single conceptual table; entity resolution between them happens at the staging stage by matching on `game_id` (the BGG canonical identifier), and the `DISTINCT ON` step picks the better-ranked row.

5 ER diagram and entity resolution

The application’s data model has two business entities (`Game` and `Review`) and a 1-to-many relationship between them: one game has zero or more reviews; each review belongs to exactly one game.

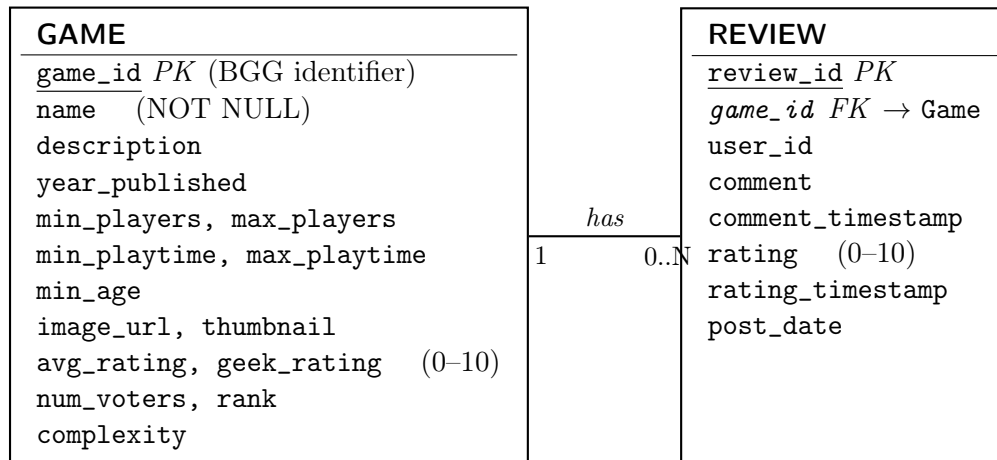


Figure 1: ER diagram. `Game 1:0..N Review`, with `Review.game_id` referencing `Game(game_id)` `ON DELETE CASCADE`.

Identification and entity resolution

- `game_id` is the BGG stable identifier — we adopted it as the primary key rather than minting a synthetic surrogate, because it remains the unique identifier across both source CSVs and external links.
- `review_id` is BGG’s review ID, also stable.
- `(game_id, user_id)` is **not** unique — the source contains multiple reviews from the same user across time.
- Entity resolution: dedup on `game_id` in `Game` (keeping the lowest-rank canonical row) and on `review_id` in `Review`; any `Review` row whose `game_id` did not survive the `Game` load is dropped via inner join, preserving referential integrity.

5.1 Note on the M2 schema vs. what we built

Our M2 proposal sketched a 9-table schema with explicit **Category**, **Mechanic**, **Designer**, **Publisher**, and **BGGUser** entities, joined to **Game** through four junction tables. We deliberately simplified this in M3 / M4 / M5 to the two-table design above for two reasons:

1. **The Kaggle datasets we ended up using do not expose category, mechanic, designer, or publisher fields on a per-game basis** — those attributes live in the BGG XML API, which is rate-limited and was out of scope to scrape for 126 k games within the project window.
2. **A single-column BGGUser(user_id) table would have a column equal to its primary key** — an anti-pattern; without any extra user-level attributes there is no functional dependency in **Review** that splitting **BGGUser** out would resolve. See also §6, “Why we did not split further”.

The two-table schema we shipped is in 3NF and BCNF; the proof is in §6 below. We discuss the implication for “Mechanic Explorer” / “Designer Leaderboard” pages in §13 (future work).

6 Schema and normalization (3NF / BCNF)

The schema lives in `sql/schema.sql` and consists of two business tables, **Game** and **Review**. Both are in **3NF** and **BCNF**. The proof is below.

6.1 Definitions used

A relation is in **3NF** if every non-trivial FD $X \rightarrow A$ has either X as a superkey or A as a prime attribute, and in **BCNF** if every such X is a superkey. A single-attribute primary key trivially satisfies 1NF and 2NF, so the analysis below focuses on which FDs hold and whether the LHS is a superkey.

6.2 Table Game

Candidate key: {game_id} only. **game_id** is the BGG canonical identifier; nothing else is guaranteed unique — **name** collides for re-implementations and **rank** only exists for ranked games. Every non-key attribute is a fact *about* that specific game and is determined exclusively by **game_id**:

```
{game_id} -> name, description, year_published,
             min_players, max_players,
             min_playtime, max_playtime, min_age,
             image_url, thumbnail,
             avg_rating, geek_rating,
             num_voters, rank, complexity
```

There are no other non-trivial FDs: **name** does not determine anything else (multiple BGG games share a title); (**year_published**, **name**) is not unique either; and the numeric BGG metrics (**avg_rating**, **geek_rating**, ...) are not deterministic functions of one another.

6.3 Table Review

Candidate key: {review_id} only. We deliberately do *not* use (**game_id**, **user_id**) as a key — the source CSV contains repeat reviews from the same user across time. The only non-trivial FDs are:

```
{review_id} -> game_id, user_id, comment,
              comment_timestamp, rating,
```

rating_timestamp, post_date

Neither `game_id` nor `user_id` determines any other attribute (one game, many reviews; one user, many reviews).

6.4 Verdict for both tables

The argument is the same for `Game` and `Review`: all attributes are atomic (**1NF**); the candidate key is a single attribute, so no partial dependency is possible (**2NF**); every non-trivial FD has the candidate key on its LHS, and the candidate key is a superkey (**3NF**); and since the LHS of every such FD is a superkey, **BCNF** holds as well. There is also no transitive dependency among non-prime attributes (e.g., `rating` does not determine `post_date`). **Both tables are in 3NF and BCNF.**

6.5 Why we did not split further

1. **Promoting `user_id` to a `User(user_id PK)` table.** The Kaggle dataset gives no other user-level data, so the resulting table would have a single column equal to its primary key — an anti-pattern. There is no FD violation in `Review` to fix.
2. **Promoting per-year aggregations to a stats table.** Aggregations like “games released in 1995 above the year average” are computed on demand in SQL (Q7); materializing them as a base table would couple the schema to a specific aggregation choice and introduce update anomalies if `Game` rows ever change. (We *do* materialize per-game review aggregates as `game_review_stats`, but as a derived performance object, not a normalized base table — see §10.)
3. **Splitting `Game` into “core attributes” + “BGG metrics”.** Both groups are functionally dependent on the same key (`game_id`), so splitting them would not satisfy any 3NF concern; it would only force more joins on every detail-page request.

7 SQL queries

10 queries are catalogued in `sql/queries.sql` and each is wired to a corresponding API route (see `docs/api.md`). The full SQL text of all ten is in Appendix A.

#	Purpose	Powers route
Q1	Top-rated games	R2
Q2	Most-reviewed games	R3
Q3	Game detail lookup	R4
Q4	Recent reviews for a game	R5
Q5	Rating distribution	R6
Q6	Highly-rated with N+ reviews	R7
Q7	Outperformers within release year (year-level aggregate CTE)	R9
Q8	Reviews exceed stored rating (materialized review summary + join)	R10
Q9	Dormant top-rated (materialized review summary + date filter)	R11
Q10	Beats global average on both axes (summary + global-stats CTE)	R12

7.1 Simple queries (Q1–Q6)

The first six queries power the catalog browsing experience:

- **Q1 (Top-rated games)** drives the “Top” rail on the Home page. Sorts `Game` by `avg_rating` DESC, `num_voters` DESC so highly-rated games with more voters surface ahead of niche outliers.
- **Q2 (Most-reviewed)** drives the “Most-reviewed” rail on the Home page. Joins `Game` with `Review`, groups by `game_id`, and orders by `COUNT(*)` DESC.
- **Q3 (Game detail lookup)** backs the entire game-detail page; it is a single-row `SELECT` on `Game(game_id)` and is the most-hit endpoint in the app.
- **Q4 (Recent reviews for a game)** returns the latest reviews for a given game, used in the right column of the detail page. See §10 for the M5 fix that aligned this query’s `ORDER BY` with the index ordering and turned a 10 s scan into a 31 ms index walk.
- **Q5 (Rating distribution)** buckets a game’s ratings into integer bins (0–10) for the histogram displayed on the detail page.
- **Q6 (Highly-rated with N+ reviews)** drives the “critically-loved” rail on the Home page; the `HAVING COUNT(*) >= N` filter prevents cult favorites from being drowned out by mainstream picks.

7.2 Complex queries (Q7–Q10)

The four bolded queries are the M4 complex queries. Each one combines multiple SQL features that a naïve single-table query cannot express:

- **Q7** originally used two correlated subqueries against `Game`. We rewrote it into a `year_stats` CTE that computes each year’s average voters and average rating once, then joins that small per-year table back to `Game`. This preserves the same idea while avoiding repeated per-row aggregation.
- **Q8** reads `game_review_stats`, the materialized one-row-per-game review summary, and joins it to `Game` to compare the review average against the stored BGG average.
- **Q9** uses the same materialized summary to find high-rated games whose precomputed `latest_post_date` is before 2024-01-01.
- **Q10** computes global averages over `game_review_stats`, then joins qualifying games to `Game`. This keeps the query complex while avoiding a fresh aggregation over all review rows.

8 API specification

13 routes total. Full spec — query params, response shape, examples — in `docs/api.md`. Quick reference:

Route id	Method/path	Purpose
R1	GET /api/games/search	Filter catalog
R2	GET /api/games/top	Top rated
R3	GET /api/games/most-reviewed	Most-reviewed
R4	GET /api/games/:gameId	Game detail
R5	GET /api/games/:gameId/reviews	Recent reviews
R6	GET /api/games/:gameId/rating-distribution	Rating histogram
R7	GET /api/games/highly-rated	Highly rated, N+ reviews
R8	GET /api/games/:gameId/similar	Similar games (heuristic)
R9	GET /api/games/outperformers-by-year	Q7 complex
R10	GET /api/games/review-vs-stored-rating	Q8 complex
R11	GET /api/games/dormant-top-rated	Q9 complex
R12	GET /api/games/global-outperformers	Q10 complex
R13	GET /api/health	Liveness

Auth: none. CORS: open (read-only public catalog). All routes return JSON.

9 Frontend pages

Page	URL	Calls
Home	/	R2 + R3 + R7 (three rails: top, most-reviewed, critically-loved)
Search	/search	R1 (filter form, server-side pagination via <code>limit/offset</code>)
Game detail	/games/:id	R4 + R5 + R6 + R8 (in parallel)
Insights	/insights	R9 + R10 + R11 + R12 (interactive controls per card)
About	/about	R13 (health) + project metadata

The whole frontend is one React SPA (`react-router-dom` v6). Styling is hand-rolled CSS with a small design system (cards, badges, table, dist bars) — no UI framework dependency.

10 Performance evaluation

Method, environment, and full numbers in `docs/performance.md`. Pipeline:

1. Capture **baseline** (`schema.sql` + `load_*.sql` only) via `sql/perf_eval.sql`.
2. Apply `sql/indexes.sql`, re-run `perf_eval.sql` for the **indexed** numbers.
3. Rewrite Q7–Q10 and re-run `perf_eval.sql` to isolate pure query restructuring effects.
4. Build `game_review_stats` with `sql/materialized_views.sql` and re-run `perf_eval.sql` for the final database-side timings.
5. Hit each of R9..R12 twice via `curl -w '%{time_total}\n'` for **cache cold/warm**.

Headline result on **126,266 Game rows** + **29,592,656 Review rows** (RDS db.t3.micro), captured in M5 with the indexes in `sql/indexes.sql` applied. After the initial M5 measurement we rewrote Q7–Q10 to test whether query restructuring could reduce cold database time beyond indexing and caching. That helped Q7 substantially but left Q8–Q10 in the 30s range, so we added `game_review_stats`, a materialized view with one precomputed row per game. The table below

reports `EXPLAIN ANALYZE` / timed-run results from `sql/perf_eval.sql` at each optimization stage.

Q	Inputs	Indexed only [†]	+ Restruct. (cold DB)	+ Mat. view (cold DB)	Warm cache
Q7	<code>limit=20</code>	2.1 s	4.1 s	0.94 s	<5 ms
Q8	<code>minReviews=100, limit=20</code>	9.0 s	29.9 s	120 ms	<5 ms
Q9	<code>minRating=8, minReviews=100, since='2024-01-01', limit=20</code>	11.8 s	34.6 s	264 ms	<5 ms
Q10	<code>limit=20</code>	9.5 s	30.5 s	151 ms	<5 ms

[†] “Indexed only” figures are M4 measurements on the 9.28 M-row `Review` sample; the “+ Restruct.” and “+ Mat. view” columns are M5 measurements on the full 29.6 M-row reload ($\sim 3.2\times$ larger). The 4.1 s / 29.9 s / 34.6 s / 30.5 s figures therefore mark the M5 cold path immediately *before* the materialized view was added. “Warm cache” is the in-process LRU hit (24 h TTL, identical inputs).

The main lesson is that query restructuring and materialization solve different problems. Q7 became fast by removing repeated work over the smaller `Game` table. Q8–Q10 needed a stronger move: precomputing `Review` aggregates once after ingestion. `game_review_stats` has 102,983 rows, compared with 29.6 M rows in `Review`, so the final queries operate on the scale of games rather than the scale of individual reviews. The LRU cache still makes repeated user interactions sub-5 ms, but the cold path is now interactive too.

Optimization techniques used

1. **Indexing** (`sql/indexes.sql`): partial B-tree on `Game.year_published`, `Game.avg_rating`, `Game.num_voters`; composite on `Review(game_id, post_date DESC)` and `Review(game_id, rating)`; partial B-tree on `Review.post_date`; GIN `pg_trgm` on `Game.name` for fuzzy/ILIKE search.
2. **Query restructuring**:
 - Q7 replaced two correlated year-level aggregate subqueries with a single `year_stats` CTE, reducing repeated work on `Game`.
 - In the intermediate Q8/Q10 rewrites, we aggregated `Review` before joining to `Game`; for Q9, we split high-rated review aggregates from recent activity and combined them with a left anti-join. These experiments clarified the bottleneck: even a cleaner plan was still paying for a full per-game review aggregation.
3. **Materialized review summary**: `sql/materialized_views.sql` creates `game_review_stats(game_id, total_reviews, rated_reviews, avg_review_rating, latest_post_date)`. Because the dataset is read-only, we build this once after `load_review.sql`. Q8–Q10 now read that 102,983-row summary instead of recomputing the same aggregate over 29.6 M reviews on every cold request.
4. **Targeted route fix**:
 - R5 (M5): tightening the predicate to `WHERE post_date IS NOT NULL` and dropping `ORDER BY ... NULLS LAST` aligned the query with the index’s native ordering, switching the planner from a Bitmap Heap Scan + top-N sort to a single Index Scan. **10.2 s** $\rightarrow$ **31 ms** ($\sim 320\times$) on game 224517. `EXPLAIN ANALYZE` before/after is in `docs/performance.md`.

5. **Application-layer cache** (`backend/cache.js`): `lru-cache v10`, `max=500`, `ttl=24 h` (M5 — bumped from 5 min because the underlying data is read-only). Keyed on route id + query params. Repeat hits never reach Postgres.
6. **Pool-level safeguards (M5)** — `backend/db.js` sets `statement_timeout = 240 s` and `query_timeout = 250 s` on every `pg` pool connection so a runaway query is aborted by Postgres and the connection is recycled instead of permanently tying up one of the 10 pool slots.

11 Local setup

Component	Notes
Database	PostgreSQL 16 on AWS RDS (<code>db.t3.micro</code> , single AZ). Hosts the data because the 29.6 M-row <code>Review</code> table is too large to keep on a laptop. Connection URL is supplied to the backend via <code>backend/.env</code> .
Backend	Node.js 20 + Express. <code>cd backend && npm install && npm run dev</code> starts the server on <code>http://localhost:8080</code> . All 13 routes go through the <code>pg</code> pool and the LRU cache.
Frontend	Vite + React 18. <code>cd frontend && npm install && npm run dev</code> serves the SPA on <code>http://localhost:5173</code> , pointed at the local backend via <code>VITE_API_BASE_URL</code> .

Liveness check: `curl http://localhost:8080/api/health` returns `{"status":"ok","db":"up"}` when the RDS connection pool is healthy.

12 Technical challenges

1. **Review CSV size.** The reviews export grew across milestones — M4 was measured on a 9.28 M-row sample; the M5 reload uses the full 29.6 M-row export. We keep raw rows in staging just long enough to do the inserts, then truncate. The expensive aggregations (Q4, Q5, Q8, Q9, Q10) are accelerated with composite indexes on `Review(game_id, ...)`. We then added the `game_review_stats` materialized view for the three Review-heavy complex routes (Q8–Q10), which brought their cold database timings down from roughly 30–35 s to 120–264 ms. We still keep the `pg` pool’s `statement_timeout` at 240 s (with a matching node-side `query_timeout` of 250 s) so unexpected slow paths cannot permanently park a pool slot.
2. **Schema simplification from M2.** Our M2 proposal designed a richer 9-table schema with `Category`, `Mechanic`, `Designer`, `Publisher`, and junction tables. The Kaggle dataset that was actually feasible to ingest at our timeline did not include those fields on a per-game basis, so we collapsed to `Game` + `Review`. This forced two downstream changes: (a) we replaced the originally-planned “Mechanics Explorer” / “Designer Leaderboard” pages with the “Insights” page over the four complex SQL queries, and (b) we had to argue 3NF/BCNF on a two-entity design — see §6.
3. **Sparse metadata in Game.** The Kaggle game export does not populate `min_players`, `max_players`, `min_age`, or `complexity`. Rather than drop those columns, we kept them in the schema (NUL-Lable) and the frontend renders them as “—”. This is documented in the schema and the ER diagram so a future iteration can pull those fields from the BGG XML API without any schema change.

4. **Separating useful query rewrites from real bottlenecks.** We rewrote Q7–Q10 late in the project and re-ran `EXPLAIN ANALYZE` rather than assuming CTEs would automatically be faster. Q7 improved substantially because the rewrite removed repeated aggregation over `Game`. Q8–Q10 improved only modestly because the dominant cost is a full aggregation over the 29.6 M-row `Review` table. That observation led directly to the `game_review_stats` materialized view, which makes the final Q8–Q10 queries operate on one row per game instead of one row per review.
5. **Frontend isolation from API URL.** We use `import.meta.env.VITE_API_BASE_URL` so the same `npm run build` artifact can be pointed at different backends (local dev, a teammate’s machine, or a future remote deployment) by changing one env var instead of editing source.
6. **AWS Academy lab volatility.** The class-issued AWS Academy sandbox occasionally times out, taking the RDS endpoint offline. We mitigated by (a) keeping the full load scripts in `sql/` so a replacement RDS instance can be re-provisioned in a few hours, and (b) caching every read route at the app layer so brief DB blips do not propagate to the user.

13 Future work

- Pull `min_players`, `max_players`, `complexity`, `mechanics`, `category`, `designer`, and `publisher` from the BGG XML API and populate the M2 lookup tables. This would unlock the originally-planned “Mechanics Explorer” and “Designer Leaderboard” pages.
- Add a `BGGUser` entity once the dataset has user-level metadata (display name, country, total reviews, etc.) — at that point a `User` table earns its keep.
- Replace the heuristic “similar games” with an embedding-based match.
- Add a “personal shelf” feature behind a real auth boundary (would be the first write path in the system).

14 Team responsibilities

Member	M3–M5 contribution
Yanru Fang	Backend (Express, all 13 routes), complex SQL queries, M4 perf evaluation.
Yuming Li	Datasets 1 & 2 cleaning (Game), Postgres schema + RDS provisioning.
Yunya Huang	Frontend Search and Game Detail flows, Similar Games component.
Fusang Leng	Dataset 3 (Reviews) cleaning, Insights page, performance evaluation.

Use of generative AI

We used Claude for two tasks on this report: LaTeX typesetting (converting our Markdown drafts into the layout used here, including the TikZ ER diagram, tables, and code listings) and grammar / clarity edits to existing prose.

A Full SQL for Q1–Q10

This appendix lists the SQL behind each of the ten queries. The listings below match `sql/queries.sql` in the repository as of the M5 reload.¹

Q1 — Top-rated games (R2)

```
SELECT game_id, name, year_published,
       avg_rating, geek_rating, num_voters, rank
FROM Game
WHERE avg_rating IS NOT NULL
ORDER BY avg_rating DESC, num_voters DESC NULLS LAST
LIMIT $1;
```

Q2 — Most-reviewed games (R3)

```
SELECT g.game_id, g.name,
       COUNT(r.review_id) AS review_count,
       AVG(r.rating)::numeric(4,2) AS avg_review_rating
FROM Game g
JOIN Review r ON r.game_id = g.game_id
GROUP BY g.game_id, g.name
ORDER BY review_count DESC
LIMIT $1;
```

Q3 — Game detail lookup (R4)

```
SELECT *
FROM Game
WHERE game_id = $1;
```

Q4 — Recent reviews for a game (R5, M5-fixed)

```
SELECT review_id, user_id, rating, comment, post_date
FROM Review
WHERE game_id = $1
      AND post_date IS NOT NULL
ORDER BY post_date DESC
LIMIT $2;
```

Q5 — Rating distribution (R6)

```
SELECT FLOOR(rating)::int AS rating_bucket,
       COUNT(*) AS review_count
FROM Review
WHERE game_id = $1
      AND rating IS NOT NULL
GROUP BY FLOOR(rating)::int
ORDER BY rating_bucket;
```

Q6 — Highly-rated with N+ reviews (R7)

```
SELECT g.game_id, g.name,
```

¹The simple queries Q1–Q6 were derived from the API spec in `docs/api.md`; the restructured Q7–Q10 match the descriptions in §10 of this report.

```

        COUNT(r.review_id) AS review_count,
        AVG(r.rating)::numeric(4,2) AS avg_review_rating
FROM Game g
JOIN Review r ON r.game_id = g.game_id
GROUP BY g.game_id, g.name
HAVING COUNT(r.review_id) >= $1
ORDER BY avg_review_rating DESC, review_count DESC
LIMIT $2;

```

Q7 — Outperformers within release year (R9, complex)

```

WITH year_stats AS (
    SELECT year_published,
           AVG(num_voters)::numeric AS avg_voters,
           AVG(avg_rating)::numeric AS avg_rating_year
    FROM Game
    WHERE year_published IS NOT NULL
           AND num_voters IS NOT NULL
           AND avg_rating IS NOT NULL
    GROUP BY year_published
)
SELECT g.game_id, g.name, g.year_published,
       g.num_voters, g.avg_rating
FROM Game g
JOIN year_stats ys
    ON ys.year_published = g.year_published
WHERE g.num_voters > ys.avg_voters
      AND g.avg_rating > ys.avg_rating_year
ORDER BY g.avg_rating DESC, g.num_voters DESC
LIMIT $1;

```

Q8 — Reviews exceed stored rating (R10, complex)

```

SELECT g.game_id, g.name,
       g.avg_rating AS stored_game_rating,
       grs.avg_review_rating AS review_avg_rating,
       grs.rated_reviews AS review_count
FROM Game g
JOIN game_review_stats grs
    ON grs.game_id = g.game_id
WHERE grs.rated_reviews >= $1
      AND g.avg_rating IS NOT NULL
      AND grs.avg_review_rating > g.avg_rating
ORDER BY (grs.avg_review_rating - g.avg_rating) DESC
LIMIT $2;

```

Q9 — Dormant top-rated (R11, complex)

```

SELECT g.game_id, g.name,
       grs.avg_review_rating AS avg_rating,
       grs.rated_reviews AS total_reviews
FROM Game g
JOIN game_review_stats grs
    ON grs.game_id = g.game_id
WHERE grs.avg_review_rating >= $1
      AND grs.rated_reviews >= $2
      AND (grs.latest_post_date IS NULL

```

```
        OR grs.latest_post_date < $3)
ORDER BY grs.avg_review_rating DESC,
        grs.rated_reviews DESC
LIMIT $4;
```

Q10 — Beats global average on both axes (R12, complex)

```
WITH global_stats AS (
    SELECT AVG(avg_review_rating)::numeric AS global_avg_rating,
           AVG(rated_reviews)::numeric AS global_avg_count
    FROM game_review_stats
    WHERE rated_reviews > 0
)
SELECT g.game_id, g.name,
       grs.rated_reviews AS review_count,
       grs.avg_review_rating
FROM Game g
JOIN game_review_stats grs ON grs.game_id = g.game_id
CROSS JOIN global_stats gs
WHERE grs.avg_review_rating > gs.global_avg_rating
      AND grs.rated_reviews > gs.global_avg_count
ORDER BY grs.avg_review_rating DESC
LIMIT $1;
```